

Claude Agent SDK

Research Agent Demo

A Deep Dive into Multi-Agent Orchestration
Architecture, Implementation & Comparison with
Perplexity, ChatGPT, and Claude Opus 4.6

Generated February 25, 2026

Source: github.com/anthropics/claude-agent-sdk-demos/research-agent

Table of Contents

1. Overview
2. Architecture Diagram
3. How It Works Step-by-Step
4. File Structure
5. The Four Agents in Detail
6. Key SDK Features
7. Observability: Hooks and Logging
8. Comparison: Perplexity, ChatGPT, Opus 4.6
9. Cost Comparison
10. Strengths and Limitations

1. Overview

The **Research Agent** is a multi-agent system built on the [Claude Agent SDK](#) (Python). It demonstrates how to orchestrate multiple specialized AI subagents that collaborate to research any topic, produce data visualizations, and generate a polished PDF report — all from a single user prompt.

What It Produces

Given a prompt like *"Research quantum computing developments in 2025"*, the system:

- Breaks the topic into 2–4 subtopics
- Runs parallel web searches across those subtopics via multiple Researcher agents
- Saves structured research notes (Markdown files packed with statistics)
- Extracts quantitative data and generates matplotlib charts (PNG)
- Synthesizes everything into a professional PDF report with embedded visuals

Output Structure

- **files/research_notes/** — Markdown files from researchers (one per subtopic)
- **files/data/** — data_summary.md from the analyst
- **files/charts/** — PNG visualizations (bar charts, line charts, etc.)
- **files/reports/** — Final PDF report
- **logs/session_*/transcript.txt** — Human-readable conversation log
- **logs/session_*/tool_calls.jsonl** — Structured tool usage log

2. Architecture

The key design insight: **the Lead Agent does zero actual work**. It only has access to the Task tool, which spawns subagents. Every piece of real work (searching, writing files, running Python scripts, creating PDFs) happens inside specialized subagents.

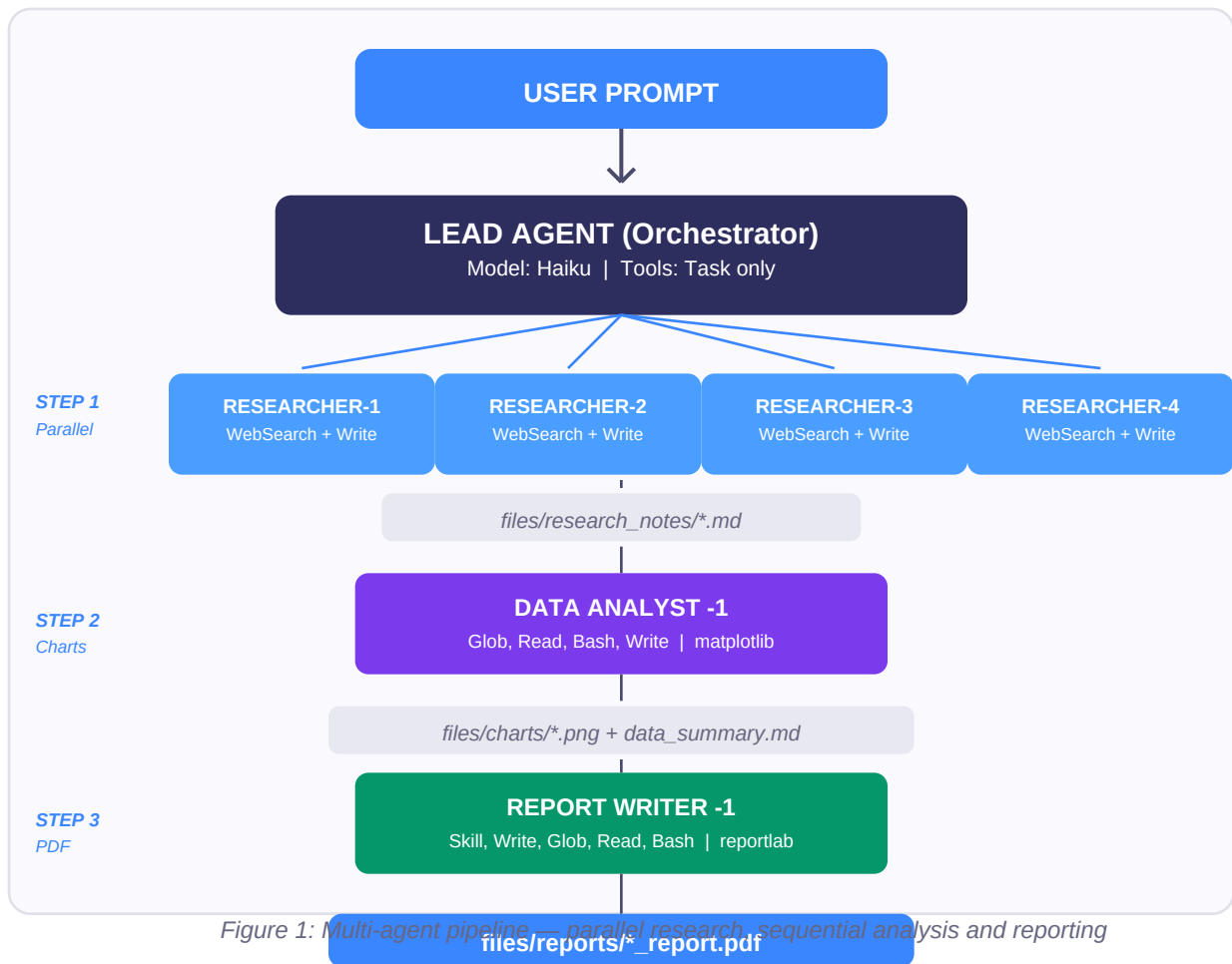


Figure 1: Multi-agent pipeline — parallel research, sequential analysis and reporting

3. How It Works Step-by-Step

Initialization

The entry point (`agent.py`) creates a `ClaudeSDKClient` configured with: `permission_mode="bypassPermissions"`, a system prompt for the Lead Agent, `allowed_tools=["Task"]`, a dictionary of 3 `AgentDefinition` objects, pre/post tool-use hooks, and `model="haiku"` for cost efficiency.

User Sends Query

The Lead Agent receives the prompt, breaks it into 2–4 subtopics (e.g., hardware/qubits, algorithms, industry/investments, challenges), and spawns 2–4 Researcher subagents in parallel via the Task tool.

Researchers Search the Web

Each Researcher has WebSearch and Write tools only. It runs 5–10 web searches with data-focused queries (adding terms like "statistics", "market size"). It extracts every number, percentage, and metric, then writes a data-rich Markdown file with 10–15+ statistics.

Data Analyst Generates Charts

Once all researchers finish, a single Data Analyst subagent reads all notes via Glob+Read, extracts quantitative data, and generates 2–4 matplotlib charts via Bash. Charts are saved as PNGs; a `data_summary.md` is written.

Report Writer Creates PDF

Finally, a Report Writer subagent reads notes, data summaries, and chart paths. It uses the "pdf" Skill for reportlab guidance, then creates a professional PDF with embedded chart images via Bash.

Completion

The Lead Agent reports: "Complete. PDF report saved to `files/reports/...`"

4. File Structure

File	Purpose
<code>agent.py</code>	Main entry point — SDK client setup, agent definitions
<code>prompts/lead_agent.txt</code>	Orchestrator instructions (176 lines, XML-structured)
<code>prompts/researcher.txt</code>	Web research specialist — data-focused search strategy
<code>prompts/data_analyst.txt</code>	Chart generation specialist — matplotlib via Bash
<code>prompts/report_writer.txt</code>	PDF creation specialist — reportlab via Bash
<code>utils/subagent_tracker.py</code>	Hook-based tool call tracking with JSONL logging
<code>utils/transcript.py</code>	Dual-output writer (console + file)
<code>utils/message_handler.py</code>	Message stream parser for subagent detection
<code>.claude/commands/*.md</code>	5 slash commands: research, competitive-analysis, market-trends, fact-check, summarize

<code>.claude/skills/pdf/</code>	PDF manipulation skill (reportlab, pypdf guides)
----------------------------------	--

5. The Four Agents in Detail

Agent	Model	Tools	Role
Lead Agent	Haiku	Task only	Decomposes topic into 2–4 subtopics, spawns and coordinates all subagents. Never does any work itself.
Researcher (x2-4)	Haiku	WebSearch, Write	Runs 5–10 data-focused web searches per subtopic. Writes Markdown notes with 10–15+ statistics, tables, and source URLs.
Data Analyst (x1)	Haiku	Glob, Read, Bash, Write	Reads research notes, extracts quantitative data, generates 2–4 matplotlib charts (PNG), writes <code>data_summary.md</code> .
Report Writer (x1)	Haiku	Skill, Write, Glob, Read, Bash	Reads notes + charts + data. Creates professional PDF via reportlab with embedded visualizations and citations.

The Lead Agent's system prompt uses XML-like tags (`<role_definition>`, `<workflow>`, `<delegation_rules>`, `<parallel_spawning>`) to enforce strict behavioral rules: always spawn researchers in parallel, never do sequential research, never skip the data-analyst step, and keep all responses to 2–3 sentences maximum.

6. Key SDK Features Demonstrated

AgentDefinition — Specialized Subagents

Each subagent is defined with a description (tells the orchestrator when to use it), a restricted tool list (principle of least privilege), a full system prompt, and a model choice. This enables clean separation of concerns.

Task Tool — Spawning Subagents

The lead agent's only tool creates new Claude instances with the specified agent definition. The SDK handles independent conversations, tool sandboxing, result return, and parallel execution when multiple Task calls are made simultaneously.

HookMatcher — Tool Call Observability

Hooks intercept every tool call before (PreToolUse) and after (PostToolUse) execution across ALL agents. The `parent_tool_use_id` links each call to its originating subagent, enabling full distributed tracing.

Skill Tool — Reusable Knowledge Packs

The `.claude/skills/pdf/` directory provides reportlab expertise that the report-writer invokes on demand. Skills deliver domain-specific knowledge without bloating every agent's system prompt.

Slash Commands — Predefined Workflows

Five commands (research, competitive-analysis, market-trends, fact-check, summarize) expand into structured prompt templates with methodology frameworks. E.g., fact-check includes a full verification process with TRUE/FALSE/PARTIALLY TRUE/MISLEADING/UNVERIFIABLE verdicts.

7. Observability: Hooks and Logging

The SubagentTracker maintains a map of sessions (`parent_tool_use_id` → SubagentSession) and tool call records (`tool_use_id` → ToolCallRecord). It auto-generates unique IDs like RESEARCHER-1, DATA-ANALYST-1. Attribution works through a chain:

1. Lead Agent spawns a researcher via Task → the ToolUseBlock has an id (e.g., "toolu_abc123")
2. `message_handler.py` detects the Task tool use and calls `tracker.register_subagent_spawn()`
3. Every subsequent AssistantMessage from that subagent includes `parent_tool_use_id = "toolu_abc123"`
4. Pre/post hooks check `_current_parent_id` to attribute tool calls to the correct subagent

This produces two log files per session: **transcript.txt** (human-readable, shows [AGENT] → Tool format) and **tool_calls.jsonl** (structured JSON Lines with timestamps, agent IDs, inputs, outputs, and success status).

8. Comparison with Perplexity, ChatGPT, and Opus 4.6

Capability	Research Agent	Perplexity	ChatGPT	Opus 4.6
Architecture	Multi-agent (4 roles)	Single-agent pipeline	Single-agent + tools	Single-agent + tools
Parallel search	2–4 simultaneous	Sequential (fast)	Sequential	Sequential
Charts	matplotlib (PNG)	None	Code Interpreter	None natively
PDF output	reportlab PDF	None	Via code only	None
Customizable	Fully (prompts, tools, hooks)	None	Limited (GPTs)	System prompts
Audit trail	Full JSONL logs	Inline citations	Inline citations	Inline citations
Self-hosted	Yes (local + API key)	No (SaaS)	No (SaaS)	No (API)

vs. Perplexity Research Mode

Perplexity is the gold standard for fast, citation-rich research. It's significantly faster (~10 seconds vs 2–5 minutes), has a purpose-built search index, and produces inline citations woven into polished web pages. However, the Research Agent can go **deeper** by design — 2–4 parallel researchers each doing 5–10 searches covers far more ground. The Research Agent is also fully programmable (change prompts, add agents, modify workflow) while Perplexity is a black box.

Use Perplexity for quick answers and daily research. Use the Research Agent for formal reports, custom workflows, and PDF/chart deliverables.

vs. ChatGPT (with Browsing & Code Interpreter)

ChatGPT browses one page at a time sequentially and runs everything in one context window, which gets crowded with long research. The Research Agent runs parallel searches across independent contexts. ChatGPT's approach is ad-hoc; the Research Agent enforces a rigid, repeatable pipeline (Research → Analyze → Report). ChatGPT's "Deep Research" (Plus/Pro) is more comparable but isn't programmable or customizable.

Use ChatGPT for interactive sessions and iterative Q&A.; Use the Research Agent for automated, reproducible report generation.

vs. Claude Opus 4.6 (Direct)

Opus 4.6 is a **more capable model** than Haiku — a single Opus query may produce higher-quality reasoning than 4 Haiku subagents combined. It's simpler (no orchestration overhead) and supports extended thinking for extraordinarily deep analysis. However, it can only search sequentially, produces text rather than file artifacts, and can't demonstrate multi-agent patterns (hooks, skills, parallel execution) that are the real value of the SDK demo.

Use Opus direct for complex reasoning and nuanced analysis. Use the Research Agent when you need structured deliverables and automated pipelines.

9. Cost Comparison (Estimated)

For a typical research query generating a 2-page PDF report:

System	Est. Cost	Est. Time
Research Agent (Haiku)	~\$0.10–0.30	2–5 minutes
Perplexity Pro	\$20/mo subscription	10–30 seconds
ChatGPT Plus (Deep Research)	\$20/mo subscription	1–3 minutes
Opus 4.6 (single query)	~\$0.15–0.50	30–90 seconds

10. Strengths and Limitations

Strengths

- **Separation of concerns** — Each agent has a focused role with restricted tool access, making the system predictable and auditable.
- **Parallel execution** — Multiple researchers run simultaneously, covering more ground faster than a single agent.
- **Full observability** — Hook system + JSONL logging provides a complete audit trail of every tool call across every agent.
- **Tangible output** — Produces real files (Markdown notes, PNG charts, PDF reports) rather than ephemeral chat responses.
- **Fully customizable** — Every aspect (prompts, tools, agent count, workflow) can be modified. Add a fact-checker agent, change chart style, swap report format.
- **Cost-efficient** — Using Haiku for all agents keeps per-token costs low while leveraging orchestration for breadth.

Limitations

- **Latency** — Sequential pipeline (research → analysis → report) takes 2–5 minutes. Each phase must complete before the next.
- **No iterative refinement** — The pipeline is one-shot. If research misses something, the report won't include it.
- **Haiku reasoning limits** — Each individual agent has less reasoning power than Sonnet or Opus. Complex analytical tasks may suffer.
- **Search quality dependency** — Report quality is bounded by WebSearch results. Bad searches → bad report.
- **No mid-flow interaction** — Once launched, the pipeline runs to completion. Can't redirect researchers or refocus the analyst.
- **Local development only** — Production deployment would need additional infrastructure, error handling, and rate limiting.

The Claude Agent SDK Research Agent demo is a reference implementation for **multi-agent orchestration**. It's not meant to replace Perplexity or ChatGPT for quick research — it demonstrates the architecture patterns that enable building custom, production-grade research systems. The real value is in the pattern, not the demo itself.

Source: github.com/anthropics/claude-agent-sdk-demos/tree/main/research-agent